Lab 3: IT567 Reinforcement Learning
202301176
Meet Barasara

1 Problem 2: Example 4.1 - 4×4 Gridworld

1.1 Problem Statement

Consider a 4×4 gridworld shown below:

gray!30T	1	2	3
4	5	6	7
8	9	10	11
12	13	14	gray!30T

Problem Parameters:

- Nonterminal states: $\mathcal{S} = \{1, 2, \ldots, 14\}$
- Terminal states: Corners (shaded cells, formally one state)
- Actions: $\mathcal{A} = \{\text{up, down, right, left}\}$ available in each state
- Transitions: Deterministic, actions that would take agent off grid leave state unchanged
- Reward: $r = -1$ on all transitions until terminal state is reached
- Task: Undiscounted, episodic ($\gamma = 1$)
- Policy: Equiprobable random policy, $\pi(a|s) = 0.25$ for all $a \in \mathcal{A}$

Transition Examples:

- $p(6, -1|5, \text{right}) = 1$ (moving right from state 5 to state 6, reward -1)
- $p(7, -1|7, \text{right}) = 1$ (moving right from state 7 stays at 7, reward -1)
- $p(10, r|5, \text{right}) = 0$ for all $r \in \mathbb{R}$ (cannot reach state 10 from 5 by going right)

Objective: Find the state-value function $v_\pi(s)$ for the equiprobable random policy, which gives for each state the negation of the expected number of steps from that state until termination.

1.2 Mathematical Formulation

1.2.1 State Space

$$\mathcal{S} = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14\} \cup \{\text{Terminal}\} \tag{1}$$

1.2.2 Action Space

$$\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\} \tag{2}$$

1.2.3 Reward Function

$$r(s, a, s') = -1 \quad \text{for all } s, s' \in \mathcal{S}, a \in \mathcal{A} \tag{3}$$

1.2.4 Bellman Equation for v_π

Since this is an undiscounted episodic task ($\gamma = 1$), the value function satisfies:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a)[r(s, a, s') + v_\pi(s')] \tag{4}$$

For the equiprobable random policy:

$$v_\pi(s) = \frac{1}{4} \sum_{a \in \mathcal{A}} [r(s, a, s') + v_\pi(s')] \tag{5}$$

where s' is the deterministic next state after taking action a from state s.

1.3 Python Implementation

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Grid dimensions
GRID_SIZE = 4

# States: 0-15 where 0 and 15 are terminal
# Mapping: state i corresponds to grid position (i//4, i%4)
TERMINAL_STATES = [0, 15]

# Actions: 0=up, 1=down, 2=left, 3=right
ACTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)]
ACTION_NAMES = ['up', 'down', 'left', 'right']
ACTION_PROB = 0.25  # Equiprobable policy

# Reward
REWARD = -1

# Convergence threshold
THETA = 1e-4

def state_to_position(state):
    """Convert state number to (row, col) position"""
    return (state // GRID_SIZE, state % GRID_SIZE)

def position_to_state(row, col):
    """Convert (row, col) position to state number"""
    return row * GRID_SIZE + col

def is_terminal(state):
    """Check if state is terminal"""
    return state in TERMINAL_STATES
```

```python

def get_next_state(state, action):
    """
    Get next state after taking action from current state
    Returns next_state and reward
    """
    if is_terminal(state):
        return state, 0

    row, col = state_to_position(state)
    action_vec = ACTIONS[action]
    next_row = row + action_vec[0]
    next_col = col + action_vec[1]

    # Check if next position is valid
    if 0 <= next_row < GRID_SIZE and 0 <= next_col < GRID_SIZE:
        next_state = position_to_state(next_row, next_col)
    else:
        # Action would take agent off grid, state unchanged
        next_state = state

    return next_state, REWARD

def iterative_policy_evaluation(gamma=1.0, theta=THETA):
    """
    Compute state-value function using iterative policy evaluation

    gamma: discount factor (1.0 for undiscounted)
    theta: convergence threshold
    """
    # Initialize value function
    V = np.zeros(GRID_SIZE * GRID_SIZE)

    # Store value functions at each iteration for visualization
    value_history = [V.copy()]

    iteration = 0
    while True:
        delta = 0
        V_new = np.copy(V)

        # Update value for each non-terminal state
        for state in range(GRID_SIZE * GRID_SIZE):
            if is_terminal(state):
                V_new[state] = 0
                continue

            v = 0
            # Sum over all actions (equiprobable policy)
            for action in range(len(ACTIONS)):
                next_state, reward = get_next_state(state, action)
                # Bellman equation
                v += ACTION_PROB * (reward + gamma * V[next_state])

            V_new[state] = v
            delta = max(delta, abs(v - V[state]))

        V = V_new
```

```python
            iteration += 1
            value_history.append(V.copy())

            print(f"Iteration {iteration}: max change = {delta:.6f}")

            # Check convergence
            if delta < theta:
                print(f"\nConverged after {iteration} iterations!")
                break

        return V, value_history

    def display_value_function(V, title="State-Value Function"):
        """Display value function as a grid"""
        V_grid = V.reshape(GRID_SIZE, GRID_SIZE)

        print(f"\n{title}")
        print("=" * 50)
        print("Grid layout (state numbers):")
        for i in range(GRID_SIZE):
            for j in range(GRID_SIZE):
                state = position_to_state(i, j)
                print(f"{state:3d}", end=" ")
            print()

        print("\nValue function v_ (s):")
        for i in range(GRID_SIZE):
            for j in range(GRID_SIZE):
                print(f"{V_grid[i, j]:6.1f}", end=" ")
            print()
        print()

    def visualize_value_function(V, value_history):
        """Visualize the value function and its evolution"""
        # Create figure with subplots
        n_snapshots = min(5, len(value_history))
        indices = np.linspace(0, len(value_history)-1, n_snapshots, dtype=int)

        fig, axes = plt.subplots(1, n_snapshots, figsize=(4*n_snapshots, 4))
        if n_snapshots == 1:
            axes = [axes]

        for idx, ax in enumerate(axes):
            V_grid = value_history[indices[idx]].reshape(GRID_SIZE, GRID_SIZE)

            # Create heatmap
            im = ax.imshow(V_grid, cmap='RdYlGn', vmin=V_grid.min(),
                           vmax=V_grid.max())

            # Add text annotations
            for i in range(GRID_SIZE):
                for j in range(GRID_SIZE):
                    state = position_to_state(i, j)
                    if is_terminal(state):
```

```
                        text = ax.text(j, i, 'T', ha="center", va="center",
                                       color="white", fontsize=12, fontweight='bold')
                    else:
                        text = ax.text(j, i, f'{V_grid[i, j]:.1f}',
                                       ha="center", va="center", color="black",
                                       fontsize=10)

            ax.set_title(f'k = {indices[idx]}', fontsize=12)
            ax.set_xticks(range(GRID_SIZE))
            ax.set_yticks(range(GRID_SIZE))
            ax.set_xticklabels([])
            ax.set_yticklabels([])
            ax.grid(True, color='black', linewidth=1)

            # Add colorbar
            plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

        plt.suptitle('Iterative Policy Evaluation: Evolution of $v_k

    \subsection{Key Insights}

    \begin{enumerate}
        \item \textbf{Poisson Process:} Models random arrival and return of cars realistically
        \item \textbf{Trade-off:} Moving costs vs. potential rental revenue
        \item \textbf{Convergence:} Policy iteration typically converges in few iterations
        \item \textbf{Dynamic Programming:} Exploits problem structure through Bellman equation
        \item \textbf{Optimal Policy:} Balances inventory across locations to maximize long-term reward
    \end{enumerate}

    \end{document},
                     fontsize=14, fontweight='bold')
        plt.tight_layout()
        plt.savefig('gridworld_4x4_value_function.png', dpi=300,
                    bbox_inches='tight')
        plt.show()

    def analyze_results(V):
        """Analyze and interpret the results"""
        print("\nAnalysis of Results")
        print("=" * 50)

        # Find states with specific characteristics
        V_grid = V.reshape(GRID_SIZE, GRID_SIZE)

        print("\nInterpretation:")
        print("-" * 50)
        print("The value function represents the negation of expected")
        print("number of steps to reach terminal state.")
        print()
        print(f"States closest to terminal: {np.argmax(V)}")
```

```python
        print(f"  Value: {V[np.argmax(V)]:.2f}")
        print(f"  Expected steps: {-V[np.argmax(V)]:.2f}")
        print()
        print(f"States farthest from terminal: {np.argmin(V[1:14])+1}")
        print(f"  Value: {V[np.argmin(V[1:14])+1]:.2f}")
        print(f"  Expected steps: {-V[np.argmin(V[1:14])+1]:.2f}")

    # Main execution
    if __name__ == "__main__":
        print("Example 4.1: 4 4  Gridworld")
        print("=" * 50)
        print("Computing v_  using Iterative Policy Evaluation")
        print("Policy: Equiprobable random (all actions equally likely)")
        print()

        # Run iterative policy evaluation
        V_pi, value_history = iterative_policy_evaluation()

        # Display results
        display_value_function(V_pi, "Final State-Value Function v_ ")

        # Analyze results
        analyze_results(V_pi)

        # Visualize evolution
        visualize_value_function(V_pi, value_history)

        print("\nVisualization saved to 'gridworld_4x4_value_function.png'")
```

1.4 Algorithm: Iterative Policy Evaluation

The algorithm computes v_π iteratively:

1. **Initialize:** $v_0(s) = 0$ for all $s \in \mathcal{S}$
2. **Iterate:** For $k = 0, 1, 2, \ldots$ until convergence:

$$v_{k+1}(s) = \frac{1}{4} \sum_{a \in \mathcal{A}} \left[r + v_k(s') \right] \tag{6}$$

   where s' is the next state after taking action a from state s

3. **Terminate:** When $\max_s |v_{k+1}(s) - v_k(s)| < \theta$

1.5 Expected Results

The final value function $v_\pi(s)$ should show:

- Terminal states: $v_\pi(\text{terminal}) = 0$
- States adjacent to terminal: approximately -1 to -2
- Center states (far from terminal): approximately -14 to -20
- Symmetry about the diagonal (due to symmetric terminal placement)

- Values represent negative expected steps to termination

Example output (approximate):

```
   0.0   -14.0  -20.0  -22.0
 -14.0   -18.0  -20.0  -20.0
 -20.0   -20.0  -18.0  -14.0
 -22.0   -20.0  -14.0    0.0
```

1.6 Key Insights

1. **Poisson Process:** Models random arrival and return of cars realistically
2. **Trade-off:** Moving costs vs. potential rental revenue
3. **Convergence:** Policy iteration typically converges in few iterations
4. **Dynamic Programming:** Exploits problem structure through Bellman equation
5. **Optimal Policy:** Balances inventory across locations to maximize long-term reward

2 Problem 3

2.1 Exercise 4.1: Computing State Values for Specific States

2.1.1 Problem Statement

In Example 4.1, if π is the equiprobable random policy, what is $q_\pi(11, \text{down})$? What is $q_\pi(7, \text{down})$?

2.1.2 Solution

Recall the action-value function:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \sum_{s',r} p(s', r|s, a)[r + \gamma v_\pi(s')] \tag{7}$$

For the 4×4 gridworld:

- $\gamma = 1$ (undiscounted)
- $r = -1$ for all transitions
- Deterministic transitions

Part 1: Computing $q_\pi(11, \textbf{down})$
From state 11 (position row 2, column 3):

- Action "down" moves to state 15 (terminal state)
- Reward: $r = -1$
- Next state value: $v_\pi(15) = 0$ (terminal)

$$
\begin{aligned}
q_\pi(11, \text{down}) &= r + \gamma v_\pi(s') && (8)\\
&= -1 + 1 \cdot v_\pi(15) && (9)\\
&= -1 + 0 && (10)\\
&= \boxed{-1} && (11)
\end{aligned}
$$

Part 2: Computing $q_\pi(7, \text{down})$
From state 7 (position row 1, column 3):

- Action "down" moves to state 11 (non-terminal)
- Reward: $r = -1$
- Next state value: $v_\pi(11) = -14$ (from Example 4.1)

$$
\begin{aligned}
q_\pi(7, \text{down}) &= r + \gamma v_\pi(s') && (12)\\
&= -1 + 1 \cdot v_\pi(11) && (13)\\
&= -1 + (-14) && (14)\\
&= \boxed{-15} && (15)
\end{aligned}
$$

2.1.3 Verification Using Python

```python
import numpy as np

# Grid parameters from Example 4.1
GRID_SIZE = 4
TERMINAL_STATES = [0, 15]

# Value function from Example 4.1 (after convergence)
# Approximate values from the book
v_pi = np.array([
    [ 0.0, -14.0, -20.0, -22.0],
    [-14.0, -18.0, -20.0, -20.0],
    [-20.0, -20.0, -18.0, -14.0],
    [-22.0, -20.0, -14.0,   0.0]
])

def state_to_position(state):
    """Convert state number to (row, col)"""
    return (state // GRID_SIZE, state % GRID_SIZE)

def get_next_state_action(state, action_name):
    """Get next state given current state and action"""
    row, col = state_to_position(state)

    action_map = {
        'up': (-1, 0),
        'down': (1, 0),
        'left': (0, -1),
        'right': (0, 1)
    }

```

```python
    d_row, d_col = action_map[action_name]
    next_row, next_col = row + d_row, col + d_col

    # Check boundaries
    if 0 <= next_row < GRID_SIZE and 0 <= next_col < GRID_SIZE:
        return next_row * GRID_SIZE + next_col
    else:
        return state  # Stay in same state if action goes off grid

def compute_q_value(state, action, v_pi, reward=-1, gamma=1.0):
    """Compute action-value q_ (s, a)"""
    next_state = get_next_state_action(state, action)
    next_row, next_col = state_to_position(next_state)
    next_value = v_pi[next_row, next_col]

    q_value = reward + gamma * next_value
    return q_value

# Exercise 4.1 calculations
print("Exercise 4.1 Solutions")
print("=" * 60)

# Part 1: q_ (11, down)
q_11_down = compute_q_value(11, 'down', v_pi)
print(f"\nq_ (11, down) = {q_11_down:.1f}")
print(f"  State 11 -> down -> State 15 (terminal)")
print(f"  = -1 + 1.0     v_ (15)")
print(f"  = -1 + 1.0     0")
print(f"  = -1.0")

# Part 2: q_ (7, down)
q_7_down = compute_q_value(7, 'down', v_pi)
print(f"\nq_ (7, down) = {q_7_down:.1f}")
print(f"  State 7 -> down -> State 11")
row_11, col_11 = state_to_position(11)
print(f"  = -1 + 1.0     v_ (11)")
print(f"  = -1 + 1.0     {v_pi[row_11, col_11]}")
print(f"  = {q_7_down:.1f}")
```

2.1.4 Summary

State	Action	Next State	$q_\pi(s,a)$
11	down	15 (Terminal)	-1
7	down	11	-15

2.2 Exercise 4.2: Modified Gridworld with State 15

2.2.1 Problem Statement

In Example 4.1, suppose a new state 15 is added to the gridworld just below state 13, and its actions, left, up, right, and down, take the agent to states 12, 9, 14, and 15, respectively. Assume that the transitions *from* the original states are unchanged. What, then, is $v_\pi(15)$ for the equiprobable random policy?

Now suppose the dynamics of state 13 are also changed, such that action down from state 13 takes the agent to the new state 15. What is $v_\pi(15)$ for the equiprobable random policy in this case?

2.2.2 Part 1: Original Dynamics for State 13

Setup:

- New state 15 added below state 13
- State 15 is non-terminal
- From state 15:
  - left $\rightarrow$ state 12
  - up $\rightarrow$ state 9
  - right $\rightarrow$ state 14
  - down $\rightarrow$ state 15 (stays in place)
- Original transitions from other states unchanged
- State 13's down action still goes to original terminal (now state 16)

Bellman Equation for State 15:

$$v_\pi(15) = \sum_a \pi(a|15) \sum_{s'} p(s'|15, a)[r + v_\pi(s')] \tag{16}$$

$$= \frac{1}{4}\Big[(-1 + v_\pi(12)) + (-1 + v_\pi(9)) + (-1 + v_\pi(14)) + (-1 + v_\pi(15))\Big] \tag{17}$$

Rearranging:

$$v_\pi(15) = \frac{1}{4}[-4 + v_\pi(12) + v_\pi(9) + v_\pi(14) + v_\pi(15)] \tag{18}$$

$$4v_\pi(15) = -4 + v_\pi(12) + v_\pi(9) + v_\pi(14) + v_\pi(15) \tag{19}$$

$$3v_\pi(15) = -4 + v_\pi(12) + v_\pi(9) + v_\pi(14) \tag{20}$$

$$v_\pi(15) = \frac{-4 + v_\pi(12) + v_\pi(9) + v_\pi(14)}{3} \tag{21}$$

Using values from Example 4.1:

- $v_\pi(12) = -22$
- $v_\pi(9) = -20$
- $v_\pi(14) = -14$

$$v_\pi(15) = \frac{-4 + (-22) + (-20) + (-14)}{3} \tag{22}$$

$$= \frac{-60}{3} \tag{23}$$

$$= \boxed{-20} \tag{24}$$

2.2.3 Part 2: Modified Dynamics for State 13

Setup:

- Now state 13's down action goes to state 15 (instead of terminal)
- This creates a system of two equations with two unknowns

Bellman Equation for State 13:
Original state 13 had:

- up $\rightarrow$ state 9: $v_\pi(9) = -20$
- left $\rightarrow$ state 12: $v_\pi(12) = -22$
- right $\rightarrow$ state 14: $v_\pi(14) = -14$
- down $\rightarrow$ state 15: $v_\pi(15) =?$

$$\begin{aligned} v_\pi(13) &= \frac{1}{4}[(-1 + v_\pi(9)) + (-1 + v_\pi(12)) \\ &\quad + (-1 + v_\pi(14)) + (-1 + v_\pi(15))] && (25) \\ &= \frac{1}{4}[-4 + (-20) + (-22) + (-14) + v_\pi(15)] && (26) \\ &= \frac{-60 + v_\pi(15)}{4} && (27) \end{aligned}$$

Bellman Equation for State 15:
From state 15:

- up $\rightarrow$ state 9: $v_\pi(9) = -20$
- left $\rightarrow$ state 12: $v_\pi(12) = -22$
- right $\rightarrow$ state 14: $v_\pi(14) = -14$
- down $\rightarrow$ state 15: $v_\pi(15)$

$$v_\pi(15) = \frac{-4 + v_\pi(12) + v_\pi(9) + v_\pi(14) + v_\pi(15)}{4} \tag{28}$$

Solving:

$$\begin{aligned} 4v_\pi(15) &= -4 + (-22) + (-20) + (-14) + v_\pi(15) && (29) \\ 3v_\pi(15) &= -60 && (30) \\ v_\pi(15) &= \boxed{-20} && (31) \end{aligned}$$

Interestingly, $v_\pi(15) = -20$ in both cases! This is because state 15 doesn't depend on state 13's value.

Finding $v_\pi(13)$:

$$\begin{aligned} v_\pi(13) &= \frac{-60 + v_\pi(15)}{4} && (32) \\ &= \frac{-60 + (-20)}{4} && (33) \\ &= \frac{-80}{4} && (34) \\ &= -20 && (35) \end{aligned}$$

2.2.4 Python Implementation

```python
import numpy as np

print("\nExercise 4.2 Solutions")
print("=" * 60)

# Values from Example 4.1
v_pi_9 = -20
v_pi_12 = -22
v_pi_14 = -14

print("\nPart 1: Original dynamics (state 13 down -> terminal)")
print("-" * 60)

# Bellman equation for state 15
# v(15) = 0.25 * [(-1 + v(12)) + (-1 + v(9)) + (-1 + v(14)) + (-1 + v(15))]
# 4*v(15) = -4 + v(12) + v(9) + v(14) + v(15)
# 3*v(15) = -4 + v(12) + v(9) + v(14)

v_pi_15_part1 = (-4 + v_pi_12 + v_pi_9 + v_pi_14) / 3
print(f"v_ (15) = (-4 + v_ (12) + v_ (9) + v_ (14)) / 3")
print(f"        = (-4 + {v_pi_12} + {v_pi_9} + {v_pi_14}) / 3")
print(f"        = {-4 + v_pi_12 + v_pi_9 + v_pi_14} / 3")
print(f"        = {v_pi_15_part1:.1f}")

print("\n\nPart 2: Modified dynamics (state 13 down -> state 15)")
print("-" * 60)

# System of equations:
# v(15) = 0.25 * [-4 + v(12) + v(9) + v(14) + v(15)]
# Solving: 3*v(15) = -4 + v(12) + v(9) + v(14)

v_pi_15_part2 = (-4 + v_pi_12 + v_pi_9 + v_pi_14) / 3
print(f"v_ (15) = (-4 + v_ (12) + v_ (9) + v_ (14)) / 3")
print(f"        = (-4 + {v_pi_12} + {v_pi_9} + {v_pi_14}) / 3")
print(f"        = {v_pi_15_part2:.1f}")

# Now compute v(13) with modified dynamics
v_pi_13_modified = (-4 + v_pi_9 + v_pi_12 + v_pi_14 + v_pi_15_part2) / 4
print(f"\nWith modified dynamics:")
print(f"v_ (13) = (-4 + v_ (9) + v_ (12) + v_ (14) + v_ (15)) / 4")
print(f"        = (-4 + {v_pi_9} + {v_pi_12} + {v_pi_14} + {v_pi_15_part2:.1f}) / 4")
print(f"        = {v_pi_13_modified:.1f}")

print("\n\nSummary:")
print("-" * 60)
print(f"Part 1: v_ (15) = {v_pi_15_part1:.1f} (state 13 unchanged)")
print(f"Part 2: v_ (15) = {v_pi_15_part2:.1f} (state 13 down -> 15)")
print(f"        v_ (13) = {v_pi_13_modified:.1f} (with modified dynamics)")
print("\nNote: v_ (15) is the same in both cases because state 15")
print("      doesn't depend on the value of state 13.")
```

2.2.5 Summary of Results

Scenario	$v_\pi(15)$	$v_\pi(13)$
Part 1: State 13 unchanged	-20	-20 (unchanged)
Part 2: State 13 down $\rightarrow$ 15	-20	-20

2.2.6 Key Insights

1. **Independence:** The value of state 15 doesn't depend on state 13's actions because state 15 only considers its own successor states
2. **Bellman Consistency:** Both states must satisfy their respective Bellman equations simultaneously
3. **Symmetry:** In the modified case, states 13 and 15 end up with the same value (-20), which makes sense given the symmetry of their positions and transitions
4. **Expected Steps:** The value -20 means approximately 20 steps expected to reach the terminal state from state 15

2.3 Problem 4

2.3.1 Exercise 4.3: Iterative Evaluation for Action-Value Functions

What are the equations analogous to (4.3), (4.4), and (4.5) for the action-value function q_π and its successive approximation by a sequence of functions $q_0, q_1, q_2, \ldots$?

2.3.2 Background

From Section 4.1, we have the following equations for state-value function v_π:

Equation (4.3) - Bellman equation for v_π:

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s] \tag{36}$$

Equation (4.4) - Expanded Bellman equation:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma v_\pi(s')] \tag{37}$$

Equation (4.5) - Iterative policy evaluation update:

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma v_k(s')] \tag{38}$$

2.3.3 Solution

The analogous equations for the action-value function $q_\pi(s, a)$ are derived by starting from the definition of q_π and following the same development as for v_π.

Analogous to (4.3) - Bellman equation for q_π:

Recall that the action-value function is defined as:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t|S_t = s, A_t = a] \tag{39}$$

Expanding the return G_t:

$$q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1}|S_t = s, A_t = a] \tag{40}$$

$$= \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1})|S_t = s, A_t = a] \tag{41}$$

Therefore, the equation analogous to (4.3) is:

$$\boxed{q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]} \tag{4.3'}$$

Analogous to (4.4) - Expanded Bellman equation for q_π:
Expanding the expectation over next states and rewards:

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\mathbb{E}_\pi[r + \gamma q_\pi(s', A_{t+1}) | S_{t+1} = s'] \tag{42}$$

$$= \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a') \right] \tag{43}$$

Therefore, the equation analogous to (4.4) is:

$$\boxed{q_\pi(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a') \right]} \tag{4.4'}$$

Analogous to (4.5) - Iterative update for q_π:
For iterative evaluation, we maintain a sequence of approximations $q_0, q_1, q_2, \ldots$ where each successive approximation is obtained using the previous one:

$$\boxed{q_{k+1}(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') q_k(s',a') \right]} \tag{4.5'}$$

for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.

2.3.4 Key Observations

1. **Structure:** The action-value equations have an extra level of summation over actions in the next state compared to the state-value equations
2. **Fixed Point:** Just as $v_k \to v_\pi$ as $k \to \infty$, we have $q_k \to q_\pi$ as $k \to \infty$ under the same convergence conditions
3. **Initialization:** $q_0(s,a)$ can be chosen arbitrarily for all s, a (except that terminal states must have $q_0(\text{terminal}, \cdot) = 0$)
4. **Relationship to v_π:** The state-value and action-value functions are related by:

$$v_\pi(s) = \sum_a \pi(a|s) q_\pi(s,a) \tag{44}$$

and

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)[r + \gamma v_\pi(s')] \tag{45}$$

2.3.5 Comparison Table

State-Value Function v_π	**Action-Value Function q_π**
(4.3) $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \| S_t = s]$	(4.3') $q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \| S_t = s, A_t = a]$
(4.4) $v_\pi(s) = \sum_a \pi(a\|s) \sum_{s',r} p(s',r\|s,a)[r + \gamma v_\pi(s')]$	(4.4') $q_\pi(s,a) = \sum_{s',r} p(s',r\|s,a) \left[r + \gamma \sum_{a'} \pi(a'\|s') q_\pi(s',a') \right]$
(4.5) $v_{k+1}(s) = \sum_a \pi(a\|s) \sum_{s',r} p(s',r\|s,a)[r + \gamma v_k(s')]$	(4.5') $q_{k+1}(s,a) = \sum_{s',r} p(s',r\|s,a) \left[r + \gamma \sum_{a'} \pi(a'\|s') q_k(s',a') \right]$

2.3.6 Python Implementation

```python
import numpy as np

def iterative_action_value_evaluation(env, policy, gamma=1.0, theta=1e-4):
    """
    Iterative evaluation for action-value function q_ (s, a)

    Parameters:
    -----------
    env : object
        Environment with state space S, action space A(s), and dynamics
    policy : dict
        Policy (a|s) - probability of taking action a in state s
    gamma : float
        Discount factor
    theta : float
        Convergence threshold

    Returns:
    --------
    Q : dict
        Action-value function q_ (s, a)
    """
    # Initialize action-value function arbitrarily
    Q = {}
    for s in env.states:
        for a in env.actions(s):
            if env.is_terminal(s):
                Q[(s, a)] = 0
            else:
                Q[(s, a)] = 0  # Arbitrary initialization

    iteration = 0
    while True:
        delta = 0
        Q_new = Q.copy()

        # Update Q(s, a) for all state-action pairs
        for s in env.states:
            if env.is_terminal(s):
                continue

            for a in env.actions(s):
                # Compute q_{k+1}(s, a) using equation (4.5')
                q_value = 0

                # Sum over all possible next states and rewards
                for s_prime, r, prob in env.transitions(s, a):
                    # Sum over all possible next actions
                    next_action_value = 0
                    for a_prime in env.actions(s_prime):
                        next_action_value += policy[(a_prime, s_prime)] * Q[(s_prime, a_prime)]

                    # Add contribution from this transition
                    q_value += prob * (r + gamma * next_action_value)
```

```python

                Q_new[(s, a)] = q_value
                delta = max(delta, abs(q_value - Q[(s, a)]))

        Q = Q_new
        iteration += 1

        print(f"Iteration {iteration}: max change = {delta:.6f}")

        if delta < theta:
            print(f"Converged after {iteration} iterations")
            break

    return Q

# Example: 4x4 Gridworld with action-value evaluation
class GridworldEnv:
    def __init__(self, grid_size=4):
        self.grid_size = grid_size
        self.states = list(range(grid_size * grid_size))
        self.terminal_states = [0, grid_size * grid_size - 1]
        self.action_map = {
            'up': (-1, 0),
            'down': (1, 0),
            'left': (0, -1),
            'right': (0, 1)
        }

    def is_terminal(self, state):
        return state in self.terminal_states

    def actions(self, state):
        return ['up', 'down', 'left', 'right']

    def state_to_position(self, state):
        return (state // self.grid_size, state % self.grid_size)

    def position_to_state(self, row, col):
        if 0 <= row < self.grid_size and 0 <= col < self.grid_size:
            return row * self.grid_size + col
        return None

    def transitions(self, state, action):
        """
        Returns list of (next_state, reward, probability) tuples
        For deterministic gridworld, probability is always 1.0
        """
        if self.is_terminal(state):
            return [(state, 0, 1.0)]

        row, col = self.state_to_position(state)
        drow, dcol = self.action_map[action]
        next_row, next_col = row + drow, col + dcol

        next_state = self.position_to_state(next_row, next_col)

        if next_state is None:
            # Hit wall, stay in same state
```

```python
            return [(state, -1, 1.0)]
        else:
            return [(next_state, -1, 1.0)]

# Create environment
env = GridworldEnv(grid_size=4)

# Define equiprobable random policy
policy = {}
for s in env.states:
    for a in env.actions(s):
        policy[(a, s)] = 0.25

# Run iterative action-value evaluation
print("Iterative Action-Value Evaluation")
print("=" * 60)
Q_pi = iterative_action_value_evaluation(env, policy, gamma=1.0, theta
    =1e-4)

# Display some results
print("\n\nSample Action-Values:")
print("-" * 60)
for s in [5, 6, 9, 10]:  # Some central states
    print(f"\nState {s}:")
    for a in env.actions(s):
        print(f"  q_  ({s}, {a}) = {Q_pi[(s, a)]:.2f}")

# Verify relationship: v(s) = sum_a   (a|s) q(s,a)
print("\n\nVerifying v(s) =  _a    (a|s) q(s,a):")
print("-" * 60)
for s in [5, 6, 9, 10]:
    v_s = sum(policy[(a, s)] * Q_pi[(s, a)] for a in env.actions(s))
    print(f"v_  ({s}) = {v_s:.2f}")
```

2.3.7 Summary

The three equations for iterative action-value function evaluation are:

1. **Bellman expectation equation (4.3'):**

$$q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

2. **Expanded form (4.4'):**

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s',a') \right]$$

3. **Iterative update rule (4.5'):**

$$q_{k+1}(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') q_k(s',a') \right]$$

These equations parallel the state-value function equations but operate on state-action pairs rather than just states, requiring an additional summation over actions in the next state.